

This part deals with the problem of word segmentation, using a simple unigram language model. Your task is to write a class `WordSegmenter` in the file `wordsegment.py`, with the following methods:

- A constructor that takes the file name of a lexicon JSON file as argument.
- `segment(text: str) -> list[str]`, which takes an unsegmented text as input and returns a list of words which maximize the value of Equation $\log P(s) = \sum_{w \in s} (\log n_w - \log N)$, where N is the total number of words in the corpus, and n_w is the corpus frequency of the word w . If you concatenate the returned words, you should get the original text back. You can assume that the text does not contain white space. The lexicon JSON file is of the following format (for a simple 3-word English lexicon): `{"a": 120.0, "an": 89.0, "ape": 2.0}`.

Although this is sufficient for producing a fairly accurate word segmentation, it is not generally a good model of language because it does not take the order of words into account. An n -gram language model conditions the probability of a word on the preceding $n-1$ words. For instance, to compute the probability of the word “a” given the context “words in”, you would divide the frequency of “words in a” by the frequency of “words in”. Intuitively this makes sense: what you are computing is simply the proportion of “words in” that are followed by “a”. One problem with this simple approach is that the numerator and denominator are often zero, leading to probabilities that are either zero (bad) or undefined (even worse). A common solution is to add a small smoothing constant k to each n -gram frequency. Write a class `NGramModel` in the file `ngram_model.py` with the following methods:

- `__init__(n: int, k: float, sentences: Iterable[tuple[str]])` that creates suitable data structures (as attributes in the `NGramModel` instance) to efficiently work with n -gram models for the given parameter, using the smoothing parameter k . The `sentences` argument is an iterable over tuples of words. That is, you can iterate once over sentences. In practice this will be data from a large text corpus.
- `p(word: str, context: tuple[str]) -> float` that returns the probability of word following after context (which has length $n-1$).
- `score(sentence: tuple[str]) -> float` that returns the log-probability of the whole sentence. That is, the sum of the (natural) logarithm of the probability of each word given its $n-1$ word context. See the ex-ample with “I am here” above for how this should be computed using the `p` method.